

Laboratório 3

Objetivos:

- Desenvolver serviços distribuídos com tecnologia *Google Remote Procedure Call* (gRPC)
- Distinguir entre chamadas unárias e chamadas com *stream* de servidor

Parte 1

- 1) Considere o exemplo dos três projetos gRPC apresentados e demonstrados na aula (anexo `grpcBase-Slides.zip`), com a estrutura apresentada na Figura 1a). Os projetos `grpcServerApp` e `grpcClientApp` têm a estrutura de diretórias Gradle com o respetivo `pom.xml`. O projeto `grpcContract`, para além do `pom.xml`, tem o contrato definido no ficheiro `serviceContract.proto` que, por convenção do compilador de protobuf, está localizado na diretória `\src\main\proto`. Veja abaixo (secção anexo) como abrir os projetos Gradle no IntelliJ.

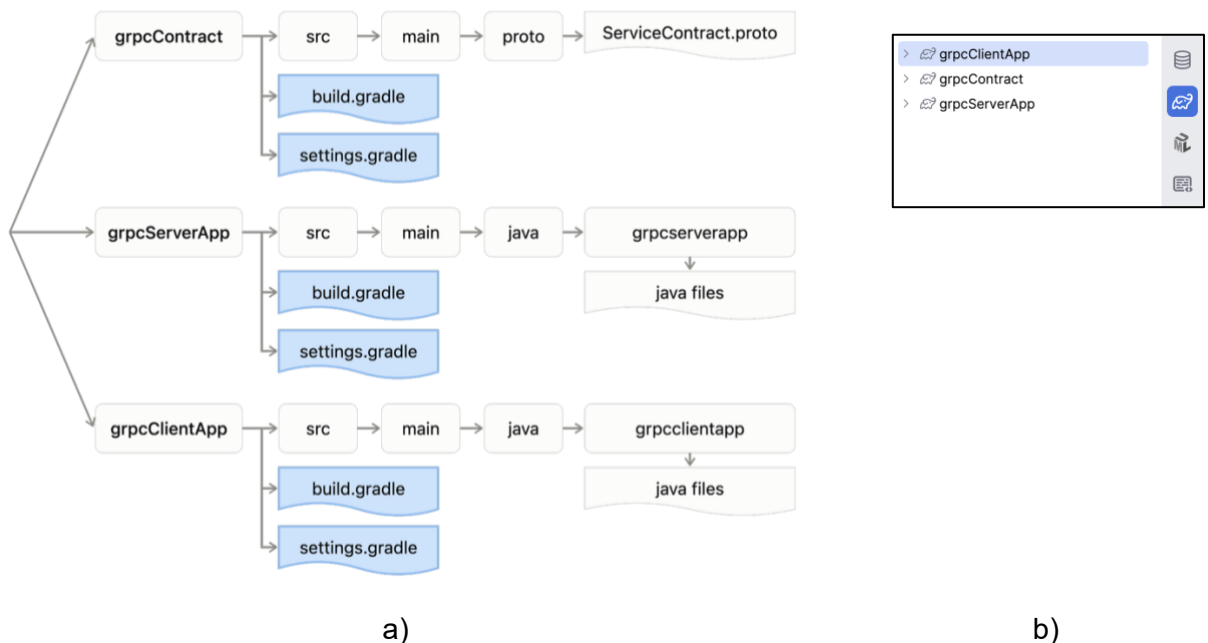


Fig. 1: a) Estrutura de diretórias e ficheiros dos projetos fornecidos; b) Janela Gradle no IntelliJ

No projeto `grpcContract`, usando o menu Gradle no ambiente IntelliJ, gere o JAR com o contrato (ação **build**).

- 2) Acrescente ao contrato a operação `divide`, que recebe dividendo e divisor e devolve quociente e resto:

```
rpc divide(DivOperands) returns (DivResult);  
  
message DivOperands { int32 dividend = 1; int32 divisor = 2; }  
message DivResult   { int32 quotient = 1; int32 remainder = 2; }
```

- a) Antes de implementar o servidor, acrescente ao cliente a opção 6 do menu, que invoca `divide`. Execute-a e observe o erro recebido. Porque é que o servidor responde, se ainda não implementou a operação?

- b) Implemente `divide` no servidor. Se o divisor for 0, o servidor deve terminar a chamada com o estado `INVALID_ARGUMENT` e uma descrição. No cliente, trate especificamente esse erro e mostre o código e a descrição do estado.
 - c) Na opção 6 do cliente, invoque `divide` com o stub síncrono e, a seguir, com o stub assíncrono. Em cada caso, mostre o resultado ou, em caso de erro, o código e a descrição do estado recebido. Onde é que o erro se manifesta em cada um dos stubs?
- 3) Acrescente ao contrato e realize a respetiva implementação no servidor da seguinte operação rpc:

```
rpc findPrimes(IntervalNumbers) returns (stream IntNumber)
message IntervalNumbers { int32 start = 1; int32 end = 2; }
```

- a) Implemente uma nova aplicação cliente que, usando um *stub* não bloqueante, realiza em paralelo no tempo, 5 chamadas à operação `findPrimes` para calcular os números primos entre 1 e 500, em intervalos de 100 números ([1,100], [101,200], ...). Note que desta forma não precisa de criar explicitamente *Threads* nem no lado do cliente, nem no lado do servidor.
- b) Após testar na sua máquina local, crie o artefacto do projeto servidor (opção *build* no menu Gradle), copie o artefacto para uma VM no seu projeto GCP e teste as aplicações cliente a conectarem-se ao servidor que está em execução na VM.